



Chat backend

The NestJS edge that authenticates users, persists conversations and adapts DIAL Core.

Backend contracts

Generation lifecycle

Port 5000

[apps/chat-api](#) · [@epam/chat-api](#)



Keep three boundaries explicit

01

Browser contract

Validated HTTP requests, DTOs and SSE responses.

02

Application services

Conversation lifecycle, auth policy and domain coordination.

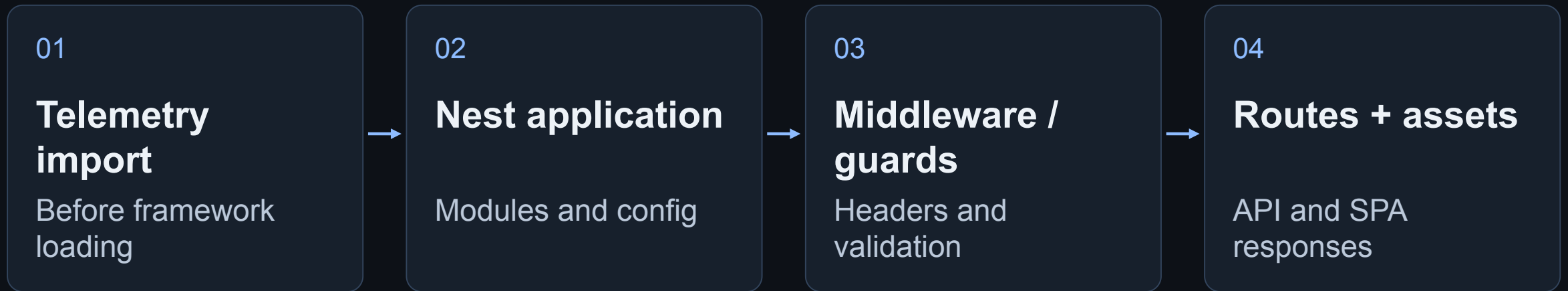
03

External systems

DIAL Core, OIDC providers, theme host and telemetry exporters.



Bootstrap establishes cross-cutting behavior



Strict ValidationPipe rejects unknown request fields.



Domain modules organize the backend

Domain family	Examples
Conversation execution	conversations, chat, client-channel
Knowledge and resources	deployments, applications, files, skills, prompts
Sharing and automation	share, publish, scheduled-tasks
Platform behavior	auth, app-config, themes, telemetry

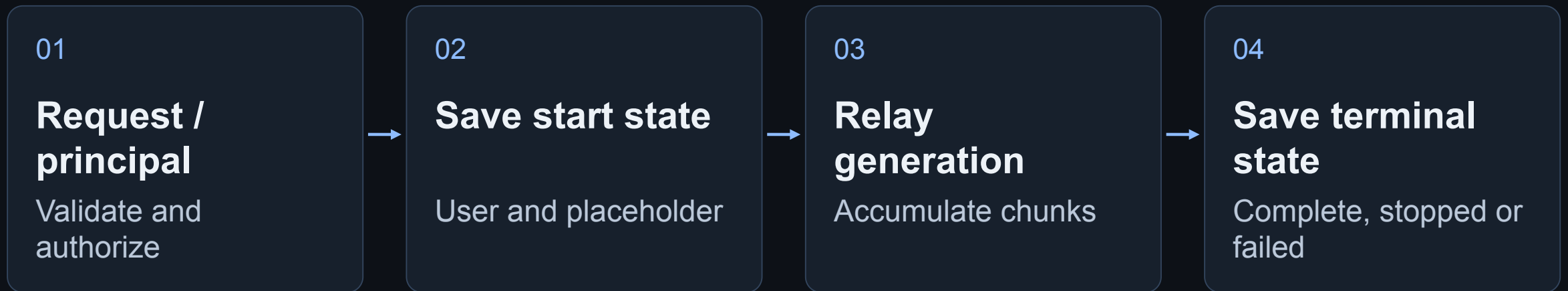


Representative HTTP contracts

Operation	Current path
List deployments	GET /api/v1/deployments
Generate a conversation	POST /api/v1/conversations/completions
Current user	GET /api/v1/auth/me
Theme configuration	GET /api/themes



A completion has a persisted lifecycle



Backend persistence survives the originating browser disconnect.

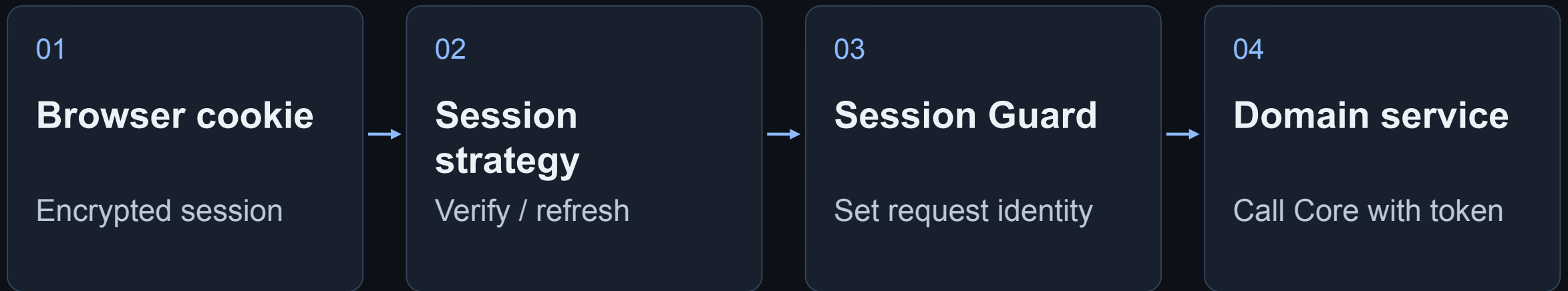


Upstream dispatch is controlled on the server

Condition	Outcome
Operator flag disabled	Use Chat Completions
Flag enabled; capability absent	Use Chat Completions
Flag enabled; responsesApi true	Use ResponsesAdapter



Authentication has explicit request scope



Header-token authentication is a separately configured strategy.



Keep controller and DTO changes synchronized

Source excerpt from ChatController · direct completion proxy.

```
@Controller({ path: 'chat', version: '1' })
export class ChatController {
  @Post('completions')
  sendCompletion(@Req() req: Request,
    @Body() dto: ChatCompletionDto) {
    const { at } = req.user as SessionUser;
    return this.chatService.sendCompletion(dto, at);
  }
}
```



Regenerate the contract after API changes



`npm run openapi` runs the complete regeneration workflow.



Configure the runtime deliberately

Configuration	Responsibility
DIAL_CORE_URL	Core transport endpoint
AUTH_* settings	Session keys and provider configuration
RESPONSES_API_ENABLED	Server-only upstream dispatch switch
MCP_APP_SANDBOX_URL	Separate-origin MCP proxy URL



Run and verify the backend

From the repository root with configured environment values.

```
npm run start:api

npm exec nx run @epam/chat-api:typecheck
npm exec nx run @epam/chat-api:test
npm run build:api

# API documentation is configured in main.ts.
# Use the deployment environment reference for auth.
```



Existing tests exercise contracts and failures

Area	Source evidence
Conversation controller	SSE, attach, errors and persistence integration specs
Authentication	Session, CSRF and strategy specs
Domain transport	Chat controller and service specs
Sandbox/static assets	App-owned asset and policy behavior tests



Operational tradeoffs shape deployment

01

Process-local state

Generation registries and caches are not durable cross-pod coordination.

02

External dependencies

Core and identity-provider availability affect user workflows.

03

Policy exceptions exist

Unversioned themes and documented transport gaps remain in the source.